Decision Trees

University of South Florida

Group: GGGamers

Group Members

Alexander Bugar
Hector Colon
Liam Dzanan
Seth Wisniewski

Program Description

Our program reads a decision tree from a file and lets the user follow its questions or browse its nodes. The terminal shows connected boxes for the nodes and highlights the current path. The included trees cover buying a car and choosing an investment. Loading a file also saves the tree's properties in `about_tree.txt`.

Instructions to Run on the Student Cluster

With CMake and a C++17 compiler installed, open a terminal in `Project 3` and run:

```
cmake -S . -B build
cmake --build build
./build/decision_tree
```

The interface uses the FTXUI files already included in `Project 2/third_party/ftxui`. Keep both project folders together, or set `-DFTXUI_DIR=/path/to/ftxui` when configuring CMake. No library download is needed.

Instructions to Interact

1. **Open a file:** Enter `tree-car.txt`, `tree-investment.txt`, or a full path, then press Enter. If the file cannot load, correct the name and try again.
2. **Walk through questions:** Use up/down and Enter to choose an answer. The choices come from the file, so they can include labels like Sometimes. A leaf shows the final decision.
3. **Explore all nodes:** Use up/down to browse nodes in preorder. Left/right moves to a parent or first child. Enter starts a walkthrough from the selected node.
4. **Move or restart:** Left arrow or Backspace goes to the parent. Press `r` to restart, Ctrl+arrows to scroll the diagram, and `c` to center it. Resizing the terminal adjusts the layout.
5. **Return or quit:** Esc returns to the menu, where another file can be opened. Press `q`, or Esc from the main menu, to quit. Use Esc to leave the filename field.

Report and Plain Console

Each successful load replaces `about_tree.txt` in the current folder. The report includes the indented tree, root, node counts, height, preorder lists, and binary tree checks. The sample files produce these results:

Property	Car	Investment
Internal nodes	4	3
Leaf nodes	6	4
Height	3	3
Binary	No	Yes
Proper	Not applicable	Yes
Perfect	Not applicable	No
Balanced	Not applicable	No

Run `./build/decision_tree --plain` for the original console. Enter a node's preorder position to see its content and one ancestor, descendant, and sibling. Missing relatives display `None`. Invalid input lets you try again, and `exit` quits. To build without FTXUI, configure CMake with `-DDECISION_TREE_UI=OFF`.

Implementation Explanation

Each linked node stores its position, edge label, content, parent, first child, and next sibling. Records are sorted by preorder position and connected using their levels. Linking siblings allows more than two children.

Recursive functions print the tree and calculate its properties. Height counts edges to the deepest leaf. A proper binary tree has two children per internal node. A perfect tree also has all leaves at one level. For balance, subtree heights differ by at most one at every node.

The display in `ui/` reads a copy of the nodes, sizes their boxes, and draws connections. The core uses only the standard library, and its destructor frees the nodes. `RUN.txt` has additional build options.

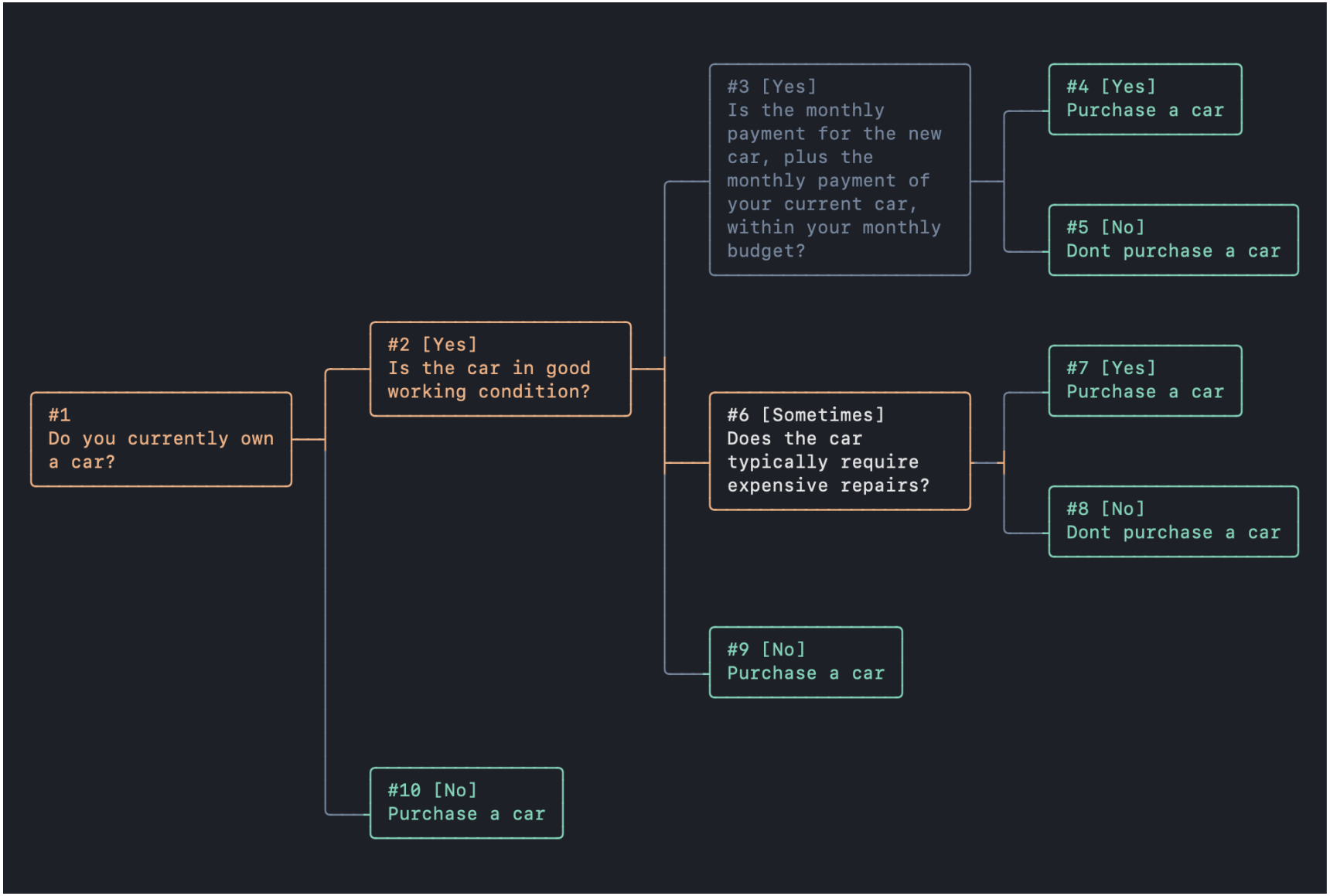
Distribution of Work

Alexander Bugar **25%**
Hector Colon **25%**
Liam Dzanan **25%**
Seth Wisniewski **25%**

Academic Integrity Statement

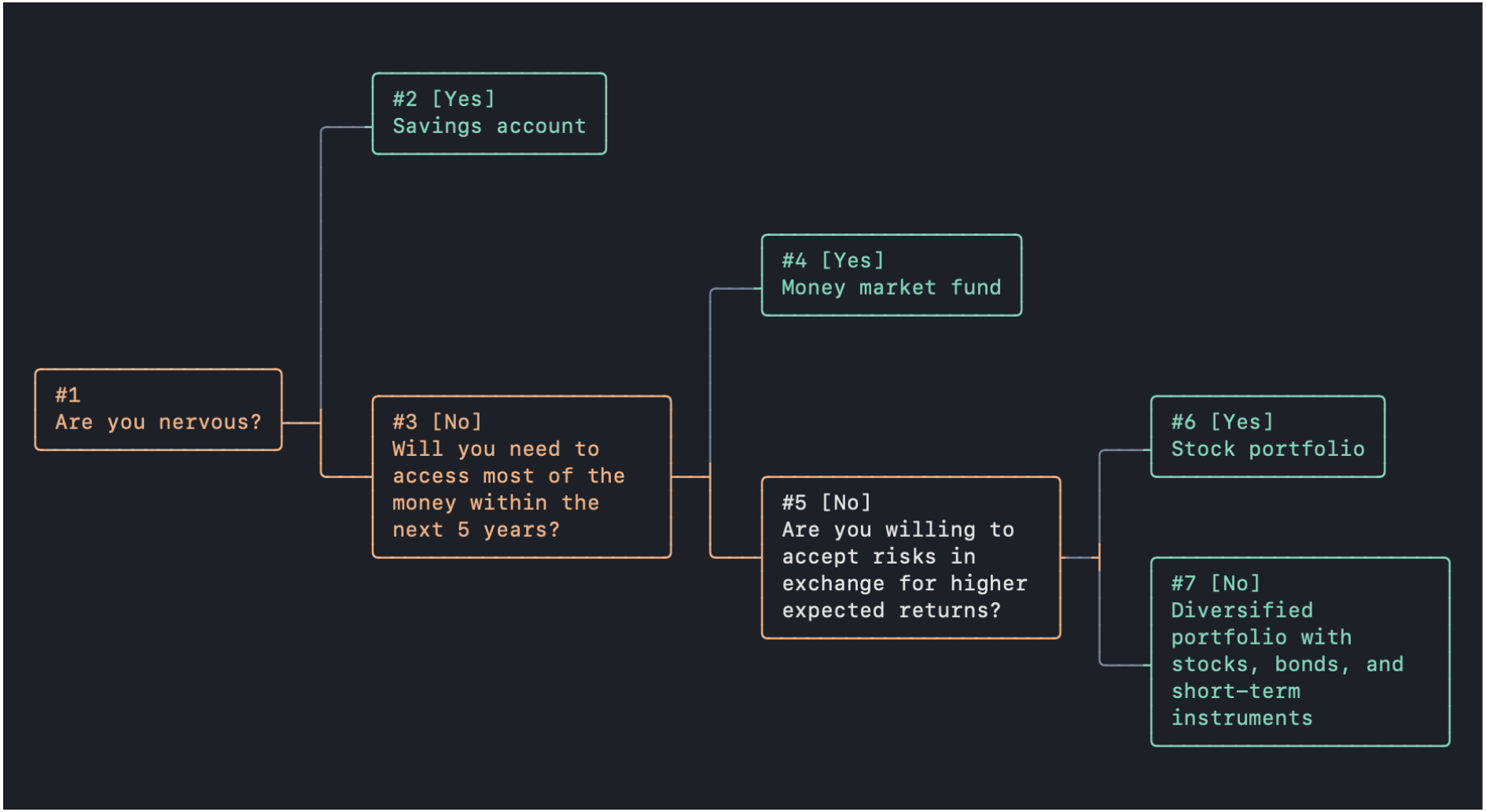
We understand that there will be no tolerance towards academic dishonesty, and that cheating will lead to an academic referral. We are aware of the identified behaviors that are considered violations of the academic standards for Undergraduate and Graduate students per USF policy.

Car Purchase Tree



All ten nodes from `tree-car.txt` are shown. Orange follows Yes and Sometimes to node 6, which asks about repairs. Green boxes are leaf nodes. Node 2 has three answers, so this is not a binary tree. Numbers match the file's preorder positions.

Investment Decision Tree



All seven nodes from `tree-investment.txt` are shown. Orange follows No twice to node 5. Each question has two answers, making the tree binary and proper. The leaves have different levels, and the root's subtree heights differ by two, so it is neither perfect nor balanced.